PROGRAMACIÓN III

Trabajo Práctico Integrador: JavaScript

OBJETIVO GENERAL

Transformar la maqueta estática del proyecto "Food Store" (desarrollada previamente en HTML y CSS) en una aplicación dinámica. El alumno deberá eliminar el contenido estático ("hardcodeado") del HTML y sustituirlo por una generación automática de elementos mediante **JavaScript**, aplicando conceptos de Arrays, Objetos, Funciones y manipulación del DOM.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
Arrays y Objetos	Simulación de una base de datos local. Se utilizará un Array de Objetos para almacenar la información de los productos (nombre, precio, imagen, etc.) y un Array de Strings para las categorías.
DOM (Document Object Model)	Selección de elementos contenedores (nodos padres) en el HTML (`getElementById` o `querySelector`) donde se inyectará el contenido.
Template Strings	Uso de comillas invertidas (`) para insertar variables de JavaScript dentro de estructuras HTML de forma limpia y legible.
Bucles (Iteradores)	Uso de métodos como `.forEach()` o `.map()` para recorrer los listados de productos/categorías y generar el HTML correspondiente por cada ítem.

Concepto	Aplicación en el proyecto
Separación de Responsabilidades	Entender que el HTML define la estructura vacía, el CSS el estilo, y JS el contenido y la lógica.

Caso Práctico

Actualmente, el archivo **`index.html`** contiene todos los productos y categorías escritos manualmente. Esto hace que actualizar el catálogo sea una tarea tediosa y propensa a errores. Se requiere automatizar este proceso mediante dos scripts principales.

```
/root
  ├── index.html
  ├── css/
  │   └── styles.css
  ├── js/
  │   ├── data.js  (Aquí irán los arrays de datos)
  │   └── main.js  (Aquí irá la lógica de renderizado)
  └── assets/      (Tus imágenes)
```

PASO 1: Preparar el HTML (`index.html`)

PASO 1: Preparar el archivo index.html

1. **Reutiliza tu base:** Toma el `index.html` que entregaste en el integrador anterior.
2. **Identifica los contenedores:** Busca la sección donde listaste tus productos y la lista de categorías.
3. **Limpia el contenido:** Borra todos los productos y categorías manuales. Deja las etiquetas "padre" (como `<main>`, `<section>` o `<ul>`) completamente vacías.
4. **Asigna IDs:** Asegúrate de que esos contenedores tengan un `id` para que JavaScript sepa exactamente dónde "escribir" la información.
   +2

Ejemplo de cómo debe quedar tu HTML:

HTML

```
<header>
    <h1>🍔 Food Store</h1>
    <nav>
        <a href="#">Inicio</a>
        <a href="#">Mis Pedidos</a>
        <a href="#">Carrito</a>
        <a href="admin.html">Panel Admin</a>
    </nav>
</header>
<aside>
    <h2>Categorías</h2>
    <ul id="lista-categorias"></ul>
</aside>
<main>
    <form>
 <input type="text" placeholder="Buscar productos...">
        <button type="submit">Buscar</button>
    </form>
    <h2>Productos Destacados</h2>
    <section id="contenedor-productos"></section>
</main>
<footer>
<p>© 2024 Food Store. Todos los derechos reservados.</p>
    <p>Contacto: info@foodstore.com</p>
</footer>
```

```
<script src="./js/data.js"></script>

<script src="./js/main.js"></script>
```

PASO 2: Definir los Datos (`js/data.js`)

En este archivo simularás la base de datos. Debes crear:

1. Un Array de Strings para las categorías.
2. Un Array de Objetos para los productos (mínimo 4 productos).

Código Guía:

JavaScript

```
// Array de categorías

const categorias = ["Hamburguesas", "Pizzas", "Papas Fritas", "Bebidas"];

const productos = [

  {

    id: 1,

    nombre: "Hamburguesa Triple",

    descripcion: "Triple carne, cheddar y bacon",

    precio: 25000,

    imagen: "https://via.placeholder.com/300x200/ff6347/ffffff?text=Hamburguesa",

    categoria: "Hamburguesas"

  },

  {

    id: 2,

    nombre: "Pizza Muzzarella",

    descripcion: "Salsa casera y orégano",

    precio: 18000,

    imagen: "https://via.placeholder.com/300x200/ff6347/ffffff?text=Pizza",

    categoria: "Pizzas"

  },

];
```

PASO 3: Lógica de Renderizado (`js/main.js`)

Desarrolla las funciones para "pintar" los datos en la pantalla.

A. Renderizar Categorías

Debes crear una función llamada cargarCategorias() que realice lo siguiente:

- Seleccione el elemento del DOM donde se insertarán las categorías (el <ul> con id "lista-categorias").
- Recorra el array de categorías definido en tu archivo de datos.
- Por cada categoría, debe crear un elemento de lista (<li>) que contenga un enlace (<a>).
- Inyecte cada nuevo elemento dentro del contenedor correspondiente.

B. Renderizar Productos

Desarrolla la función cargarProductos() siguiendo estos requerimientos técnicos:

- **Iteración:** Utiliza un método de array (como forEach) para procesar tu lista de productos.
- **Creación de Estructura:** Por cada producto, debes generar un nodo <article> y asignarle la clase CSS necesaria para que mantenga el estilo de tu proyecto anterior.
- **Uso de Plantillas:** Utiliza **Template Strings** (comillas invertidas ` `) para maquetar el contenido interno del article.
- **Contenido Dinámico:** La estructura debe incluir la imagen, el nombre, la descripción y el precio, accediendo a las propiedades del objeto producto.
- **Interactividad:** El botón de "Agregar" debe mostrar un mensaje (puedes usar un alert) que indique el nombre del producto seleccionado.

Requisitos:

- **Variables:** Utilizar `const` para declarar arrays y funciones.
- **Semántica:** El HTML generado por JavaScript debe respetar las etiquetas `<article>`, `<h3>`, `<p>` y `<button>` definidas en el TP de HTML.
- **Rutas:** Las imágenes deben visualizarse correctamente. Recuerda que las rutas relativas en el array de productos se resuelven desde la ubicación del `index.html`.
- **Validación:** No debe haber errores en la consola del navegador (F12).

CONCLUSIONES ESPERADAS

Al finalizar este Trabajo Práctico, se espera que el alumno haya logrado:

- **Comprender la separación de responsabilidades:** Entender que el HTML define la estructura, el CSS la apariencia y JavaScript el contenido y el comportamiento.
- **Independizar los datos de la vista:** Asimilar que la información (productos, categorías) no debe estar "atada" al código HTML, sino residir en estructuras de datos (Arrays/Objetos) que permiten su fácil modificación y escalabilidad.
- **Dominar el flujo de renderizado:** Visualizar cómo JavaScript actúa como intermediario, tomando datos crudos y transformándolos en elementos visuales del DOM (Nodos HTML) en tiempo de ejecución.
- **Automatizar tareas repetitivas:** Valorar la utilidad de los bucles (`forEach`) para generar múltiples elementos repetitivos (como tarjetas de productos) escribiendo el código una sola vez, reduciendo errores y facilitando el mantenimiento.
- **Preparar la lógica para el futuro:** Dejar el proyecto listo para, en la siguiente etapa, reemplazar el archivo `data.js` local por una petición real a una API externa sin tener que modificar la estructura visual del sitio.